

CHECKLIST HOÀN THIỆN ĐỀ TÀI — TRIỂN KHAI LẠI TỪ ĐẦU

Docker vừa reset, mất toàn bộ data cũ — coi như triển khai lại từ đầu. Danh sách gồm 2 phần: (A) Dựng lại toàn bộ hạ tầng + chạy thử, (B) Việc còn thiếu để hoàn thiện đề tài.

PHẦN A — TRIỂN KHAI LẠI TỪ ĐẦU

A1. Hạ tầng Docker

- ☒ Kiểm tra docker-compose.yml có đủ 4 service: postgres_db, mysql_db, mongo_db, clickhouse_db + service mongo_init
- ☒ clickhouse_db PHẢI có environment CLICKHOUSE_PASSWORD (bản ClickHouse mới bắt buộc, không để trống được)
- ☒ network khai báo driver: bridge (không dùng external: true trừ khi có stack CDC khác cần nối chung)

```
docker compose up -d
docker compose ps          # xem 4 container có chạy không
docker compose logs mongo_init # kiểm tra rs.initiate() thành công
```

- ☒ Xác nhận cả 4 container status "running", mongo_init log có {ok: 1}

A2. Môi trường Python (venv)

```
cd D:\schema-evolution-core
python -m venv venv
.\venv\Scripts\Activate.ps1
pip install -r requirements.txt
```

- ☒ requirements.txt gồm: psycopg2-binary, mysql-connector-python, PyYAML, requests, streamlit, pymongo, clickhouse-connect

A3. Khởi tạo dữ liệu mẫu (chạy lại vì Docker mất data)

- ☐ Postgres (source pg_to_mysql): chạy init_source.sql qua DBeaver — tạo bảng customers, orders
- ☐ MySQL (target pg_to_mysql): chạy LẠI init_source.sql y hệt — target phải khớp source từ đầu
- ☐ ClickHouse (target mongo_to_clickhouse): chạy init_clickhouse.sql — tạo bảng users
- ☐ MongoDB (source mongo_to_clickhouse): chạy init_mongo_source.js qua Compass Shell
- ☐ (Tuỳ chọn) pair mysql_self_monitor dùng chung data MySQL đã có, không cần thêm gì

A4. Cấu hình config.yaml

- ☐ registry.dir — đường dẫn lưu Registry (VD ./data/registry)
- ☐ scheduler.interval_seconds — chu kỳ quét (VD 60)
- ☐ telegram.bot_token / chat_id — dùng token MỚI (token cũ đã revoke)
- ☐ pairs: pg_to_mysql (source postgres, target mysql, tables: customers, orders)
- ☐ pairs: mongo_to_clickhouse (source mongodb, target clickhouse, tables: users)
- ☐ (Tuỳ chọn) pairs: mysql_self_monitor (source mysql, KHÔNG có target)

A5. Kiểm tra đủ file trong project

File	Vị trí
base.py, postgres.py, mysql.py, mongo.py, clickhouse.py	src/schema_evolution/engines/
core.py, notification.py	src/schema_evolution/
scheduler.py, app.py, unfreeze_cli.py, config.yaml	app/

requirements.txt	gốc project
init_source.sql, init_target.sql, init_clickhouse.sql	docker/ (hoặc tùy ông đặt)
init_mongo_source.js, init_mongo_drift.js	docker/ (hoặc tùy ông đặt)

A6. Chạy và kiểm tra end-to-end

- ☐ Chạy scheduler.py — kỳ vọng mọi pair/bảng ra "initialized" (không lỗi kết nối)
- ☐ Test non-breaking: ADD COLUMN trên Postgres source → đợi 1 chu kỳ → MySQL target tự có cột mới + Telegram AUTO-SYNC
- ☐ Test breaking: DROP COLUMN trên Postgres source → freeze + draft + Telegram SCHEMA DRIFT
- ☐ streamlit run app.py → bấm Approve → MySQL target THỰC SỰ được DROP COLUMN theo (approve_change_and_sync) + Telegram ĐÃ PHÊ DUYỆT
- ☐ Test Reject tương tự → Registry giữ nguyên baseline cũ, hệ thống phát hiện lại ở chu kỳ sau
- ☐ Test tương tự cho mongo_to_clickhouse: chạy init_mongo_drift.js → ClickHouse tự thêm cột Nullable(...)
- ☐ Kiểm tra tab "Lịch sử phiên bản" hiện đúng version + mốc thời gian

PHẦN B — VIỆC CÒN THIẾU ĐỂ HOÀN THIỆN ĐỀ TÀI

B1. Sandbox backend (ưu tiên cao nhất — frontend Schema Editor đang cần)

- ☐ Thêm 2 method vào BaseEngine: clone_table_structure(table_name, sandbox_name), drop_table(table_name)
- ☐ Implement cho PostgresEngine, MySQLEngine, ClickHouseEngine
- ☐ MongoEngine: không cần implement thật (no-op sẵn)
- ☐ Thêm test_changes_in_sandbox(engine, table_name, changes) vào core.py — clone, áp DDL thử, bắt lỗi, LUÔN xóa bảng nháp (finally)
- ☐ Nối vào app.py: nút "Test trước khi áp dụng" cạnh nút Approve
- ☐ Nối vào Schema Editor frontend (đã build) — expose đúng hàm/API
- ☐ Test kỹ: sandbox FAIL đúng cách khi DDL sai, và LUÔN dọn dẹp dù pass hay fail

B2. Task / Flow Monitor Dashboard

- ☐ Thêm trang tổng quan trong app.py: liệt kê TẤT CẢ pair/bảng, trạng thái, version, lần quét gần nhất
- ☐ Dữ liệu lấy từ Registry có sẵn (is_frozen, version, updated_at) — không cần bảng lưu mới
- ☐ (Tùy chọn) bộ lọc theo trạng thái (đang frozen / bình thường)

B3. Schema Editor — nối frontend với backend

- ☐ Frontend gọi: dựng TableSchema mới từ input → compare_schemas(DB thật, schema mới) → hiện preview
- ☐ Nút "Test" gọi test_changes_in_sandbox() (B1)
- ☐ Nút "Áp dụng" gọi generate_ddl + execute_ddl (hoặc approve_change_and_sync nếu qua breaking flow)
- ☐ Test thử với non-IT thật nếu có thể

B4. Debezium Event-driven (ưu tiên thấp nhất, để sau)

- ☐ Cấu hình Debezium connector bắt DDL, ghi vào Kafka topic schema-changes
- ☐ Viết Kafka consumer nhỏ — nghe topic, có message thì gọi run_evolution_check() ngay
- ☐ Giữ nguyên polling định kỳ song song làm lưới an toàn

- ☐ Đo thử độ trễ phát hiện: event-driven vs polling — làm minh chứng báo cáo

B5. Báo cáo (song song, không cần chờ code xong hết)

- ☐ Thống nhất thuật ngữ Schema Registry xuyên suốt Chương II/III
- ☐ Cập nhật Chương III: thêm Airbyte + Debezium Schema History vào bảng so sánh (không thêm SQLMesh)
- ☐ Sửa đoạn 3.4: bổ sung câu Approve tự động đồng bộ DDL lên target
- ☐ Sửa diagram: thêm mũi tên Approval Workflow → Data Consumer, sửa Data Consumer thành MySQL / ClickHouse
- ☐ Viết Chương IV dựa trên hệ thống ĐÃ CHẠY THẬT — không viết trước
- ☐ Chụp ảnh/quay demo thật (Telegram, Streamlit UI, Approve/Reject) cho báo cáo/slide bảo vệ

Lưu ý xuyên suốt: mỗi bước code mới nên test bằng mock trước, sau đó mới test với Docker thật — tránh lặp lại các lỗi đã gặp (thiếu abstract method, sai cú pháp SQL, quên rollback connection, autocommit MySQL...).